Tecnológico Nacional de México
Instituto Tecnológico El Llano
Aguascalientes

Maestría en Inteligencia Artificial

Aprendizaje Automático Avanzado

Unidad 2. Aprendizaje por Refuerzo Profundo

Docente

Dr. Iván Castillo Zúñiga

¡Bienvenidas y bienvenidos a la Unidad 2!

El aprendizaje por refuerzo profundo estudia cómo un agente aprende mediante la interacción con su entorno y las consecuencias de sus decisiones.

Pregunta detonadora

¿Cómo aprende una máquina cuando nadie le proporciona la respuesta correcta en cada paso?

Al finalizar esta parte, el estudiante comprenderá los elementos fundamentales, el modelo de Markov y el funcionamiento de DQN.

Instituto Tecnológico El Llano Aguascalientes

Contenido

2.1 Principios del aprendizaje por refuerzo
2.2 Diseño de agentes inteligentes que aprendan empleando DQN y PPO
2.3 Impacto ético y técnico del aprendizaje por refuerzo

Maestría en Inteligencia Artificial

Elementos del aprendizaje por refuerzo

Agente
Toma decisiones.

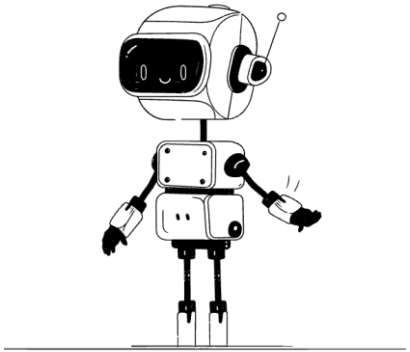
Entorno
Espacio donde actúa.

Estado
Información disponible en un momento.

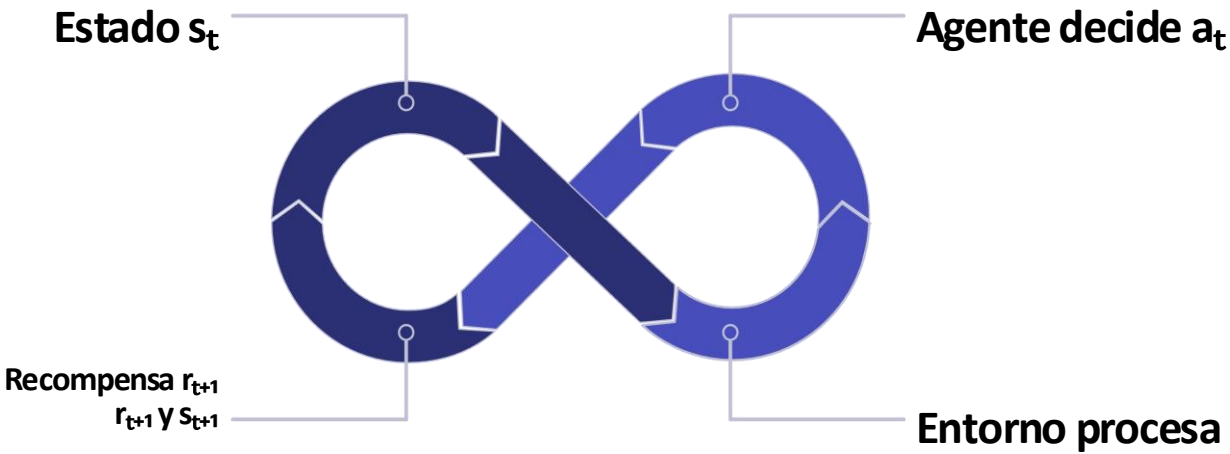
Acción
Decisión ejecutada.

Recompensa
Señal que orienta el aprendizaje.

Política
Estrategia para seleccionar acciones.



Ciclo agente - entorno



El agente observa, actúa, recibe retroalimentación y actualiza su estrategia.

Ejemplo: Recompensa positiva al acercarse a una meta y penalización por colisionar

Proceso de decisión de Márkov



La propiedad de Markov indica que el siguiente estado depende únicamente del únicamente del estado y la acción actuales. actuales.

Retorno acumulado

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

γ determina la importancia de las recompensas futuras ($0 \leq \gamma \leq 1$).

Exploración frente a Explotación



Estrategia ε-greedy

Con probabilidad ε se explora; con $1-\varepsilon$ se selecciona la mejor acción conocida.

Ejemplo: un sistema de reparto prueba ocasionalmente rutas distintas.

Probar acciones nuevas para descubrir mejores alternativas.

Utilizar la mejor acción conocida para maximizar la recompensa inmediata.

1.1 Principios del aprendizaje por refuerzo

Para ilustrar estos conceptos, utilizaremos Q-Learning, uno de los algoritmos más fundamentales del aprendizaje por refuerzo. Lo aplicaremos al entorno FrozenLake de la librería gymnasium.

En este problema, un agente debe cruzar un lago congelado (una cuadrícula de 4x4) desde el inicio hasta una meta sin caer en agujeros.

Para ejecutar el siguiente código, necesitarás instalar la librería: pip install gymnasium numpy.

```
#Implementación en Python

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

# 1. Configuración
env = gym.make("FrozenLake-v1", is_slippery=False)
q_table = np.zeros((env.observation_space.n, env.action_space.n))

# Hiperparámetros
learning_rate = 0.1
gamma = 0.99
epsilon = 1.0
epsilon_decay = 0.001
episodes = 2000

# Lista para almacenar el historial de recompensas
rewards_per_episode = []

# 2. Entrenamiento
for i in range(episodes):
  state, _ = env.reset()
  done = False
  total_rewards = 0

  while not done:
    # Selección de acción (Epsilon-greedy)
    if np.random.uniform(0, 1) < epsilon:
      action = env.action_space.sample()
    else:
      action = np.argmax(q_table[state, :])

    new_state, reward, terminated, truncated, _ = env.step(action)
    done = terminated or truncated

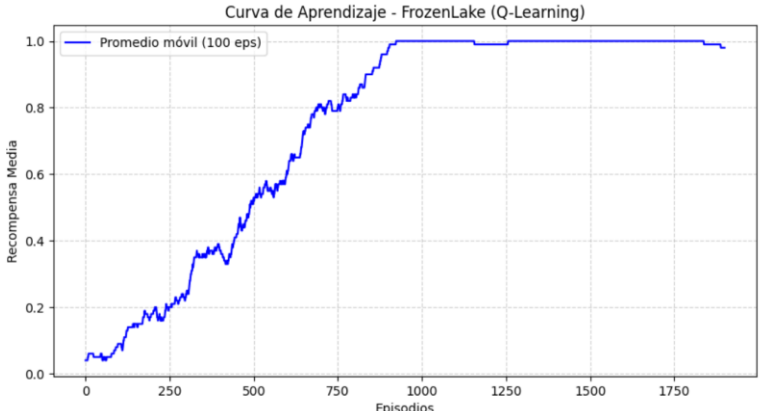
    # Actualización de Q-Value
    q_table[state, action] = q_table[state, action] + learning_rate * (
      reward + gamma * np.max(q_table[new_state, :]) - q_table[state, action]
    )

    state = new_state
    total_rewards += reward

  epsilon = max(0.01, epsilon - epsilon_decay)
  rewards_per_episode.append(total_rewards)

# 3. Cálculo del promedio móvil (cada 100 episodios)
# Esto suaviza la curva para ver la tendencia de aprendizaje
rolling_length = 100
average_reward = []
for i in range(len(rewards_per_episode) - rolling_length):
  average_reward.append(np.mean(rewards_per_episode[i:i+rolling_length]))

# 4. Graficación
plt.figure(figsize=(10, 5))
plt.plot(average_reward, color='blue', label='Promedio móvil (100 eps)')
plt.title('Curva de Aprendizaje - FrozenLake (Q-Learning)')
plt.xlabel('Episodios')
plt.ylabel('Recompensa Media')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```



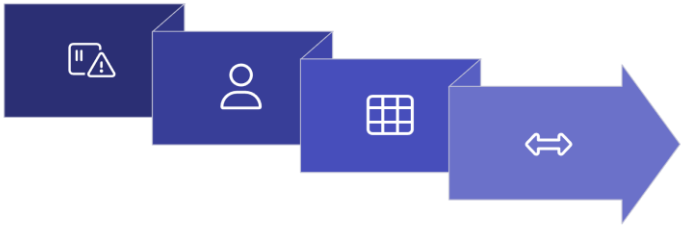
1. **Fase Inicial (Exploración):** Al principio (episodios 0-500), la recompensa media es cercana a 0. El agente está explorando aleatoriamente y rara vez llega a la meta.
2. **Fase de Ascenso (Aprendizaje):** La curva comienza a subir. Aquí, el agente ha descubierto rutas exitosas y, gracias a que la **función de valor (Q)** se está actualizando, empieza a preferir esas rutas sobre el movimiento al azar.
3. **Fase de Convergencia (Explotación):** Hacia el final, la gráfica se estabiliza cerca de 1.0. Esto indica que el agente ha encontrado la **política óptima (π*)** y es capaz de resolver el entorno de manera consistente.

De Q-learning a DQN

Idea central: DQN aproxima $Q(s, a)$ mediante una red neuronal profunda, superando las limitaciones de la tabla Q clásica.

Aspecto	Q-learning	DQN
Representación	Tabla Q	Red neuronal
Estados	Discretos y reducidos	Numerosos o complejos
Generalización	Limitada	Entre estados semejantes
Entrada	Identificador	Variables, sensores o imágenes

Arquitectura de una Deep Q NetWork



9·≔E Ï ′ⓃⓋ◇éƛtNⓆ⁂俥9Ⓩƛé◇¡ ◇α

- Posición del carro
- Velocidad del carro
- Ángulo de la barra
- Velocidad angular de la barra

Acciones y objetivo

- Mover a la izquierda
- Mover a la derecha

Objetivo: mantener la barra equilibrada el mayor tiempo posible.

Entrenamiento de una DQN



Memoria de repetición

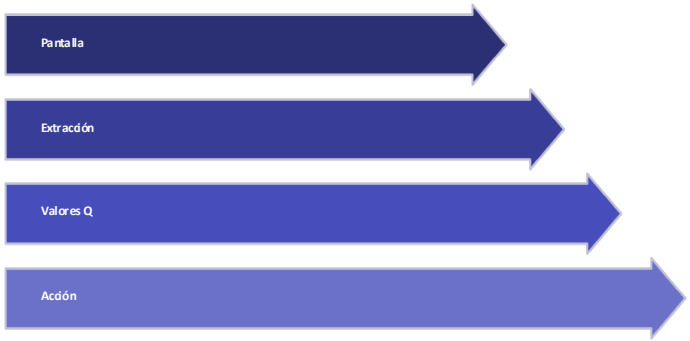
Reduce correlaciones entre experiencias consecutivas.

Red objetivo

Ï ØÐÓÖØÁÑÒÐÕMQMÕÐØÁÑCE
ÑÑ ØÁÑ,ÑØÁÑÕÑÒÁÒÐM
ÑCEMNÕÑCEÑÞØÁMÐPÑ ÑÕ
ÑÕPØÁÑÕMÖ ÐÑÕPÖB

Material: Ejemplo – Pagina 13 - DQN es ideal para entornos con espacios de acción discretos (como mover a la izquierda o derecha). En este ejemplo, entrenaremos a un agente para equilibrar un poste sobre un carro (CartPole-v1).

Caso clásico: Video juego Atari



Reto

Aprender acciones útiles directamente desde píxeles y recompensas.

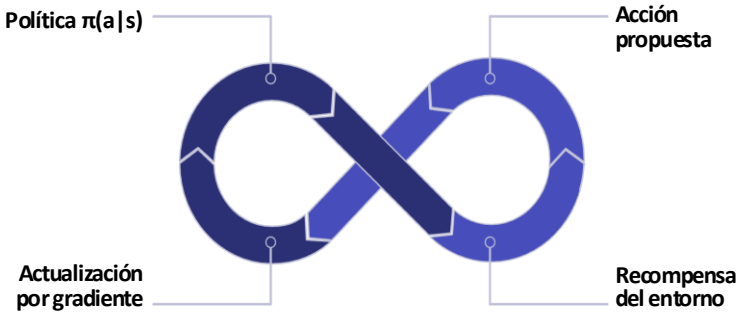
Enfoque

Red convolucional + DQN + + memoria de repetición + red + red objetivo.

Aportación

Integra percepción, decisión y decisión y aprendizaje en una una misma arquitectura.

Métodos basados en política



PPO es más versátil y estable. Es un algoritmo de Actor-Crítico que funciona muy bien tanto en espacios de acción discretos como continuos. Ejemplo en su material – **página 15.**

Optimización directa

Optimizan directamente una política parametrizada.

Políticas estocásticas

Admiten distribuciones de probabilidad sobre acciones.

Acciones continuas

Se adaptan a espacios de acciones acciones continuas como velocidad o velocidad o dirección.

Alta varianza

Pueden presentar alta varianza varianza durante el entrenamiento

Ejemplo: controlar continuamente la velocidad, altura y dirección de un dron. dron

Algoritmos representativos

Algoritmo	Enfoque	Acciones	Fortaleza
DQN	Basado en valor	Discretas	Estados complejos (videojuegos)
PPO	Basado en política	Discretas / continuas	Actualizaciones estables (robótica)
DDPG	Actor-crítico	Continuas	Control determinista (navegación)
SAC	Actor-crítico	Continuas	Exploración por entropía (control industrial)

3.2 Impacto ético y técnico

Cuando trasladamos estos algoritmos del código a la realidad física o social, surgen interrogantes críticas.

Síntesis del Impacto en Sectores Clave

Sector	Impacto Técnico Primario	Desafío Ético Crítico
Robótica	Estabilidad de control y eficiencia de muestreo.	Seguridad física y responsabilidad legal ante fallos.
Videojuegos	Generación de comportamiento humanoide y equilibrio de juego.	Manipulación del comportamiento del usuario y adicción.
Sistemas Autónomos	Navegación en entornos no estructurados.	El dilema del "tranvía" y la toma de decisiones morales programadas.

Actividades

1. Agente DQN para Entornos Discretos (Atari/Videojuegos) – **Programa de generación de agentes inteligentes**

2. **Foro** - El impacto ético y técnico del aprendizaje por refuerzo en entornos complejos como la robótica y los video juegos

Castillo-Zúñiga, I. et al. (2022). Clasificación de características asociadas al cibersuicidio. [Referencia incluida en el documento base].

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.